

«cnt_en_clr» (счётчик с разрешением на запись и очистку)

Старостин Александр, Б01-401

starostin.aa@phystech.edu

Вариант 5

23 марта 2026 г.

Содержание

1	Вступление	3
2	Документация разрабатываемого модуля	3
2.1	Функциональное описание	3
2.2	Структурная схема	4
2.3	Параметры	4
2.4	Порты	4
2.5	Тактирование и сброс	5
2.6	Тестирование	5
3	Скачивание и запуск кода	5
4	Файл с имплементацией разработанного модуля	6
5	Файлы с testbench для разработанного модуля	7
5.1	Файл с testbench для всего функционала модуля, кроме сигнала переполнения	7
5.2	Файл с testbench для переполнения	12
6	Временные диаграммы	16
6.1	testbench для всего функционала модуля, кроме сигнала переполнения . . .	16
6.2	Файл с testbench для переполнения	17
7	Вывод	18

1 Вступление

В данном документе приведён отчёт о разработанном модуле `cnt_en_clr` (counter enable clear). Это счётчик с сигналами разрешения счёта и очистки. Код написан на языке Verilog. В качестве верификации использовались временные диаграммы. Для получения диаграмм код компилировался и запускался с помощью программы «Icarus Verilog». Просмотр временных диаграмм будет производиться в программе «GTKWave».

2 Документация разрабатываемого модуля

В данном разделе приведена документация к разрабатываемому модулю. Документация включает в себя следующие разделы: функциональное описание, структурная схема, параметры, порты, тактирование и сброс, тестирование.

2.1 Функциональное описание

Опишем функционал разрабатываемого модуля.

Модуль «`cnt_en_clr`» – это счётчик, который меняет своё значение по положительному фронту такового сигнала (увеличивает значение на 1, при выставленном сигнале разрешения записи; обнуляется при выставленном сигнале очистки (очистка приоритетнее, чем запись); в противном случае - сохраняет своё значение) и по отрицательному фронту сигнала сброса (сигнал сброса приоритетнее, чем тактовый сигнал).

Размер счётчика может варьироваться с помощью параметра.

Кроме самого значения счётчика хранится информация о переполнении счёта. Для этого имеется сигнал переполнения. К примеру, если счётчик 4-х битный и до положительного фронта тактового сигнала имеет значение 1111 в двоичной СИ (сигнал переполнения выставлен в 0), то после положительного фронта (при разрешении на запись и запрете на очистку) счётчик станет хранить значение 0000, а сигнал переполнения – 1. Повторное переполнение не обрабатывается, т.е. при повторном переполнении счётчика сигнал переполнения станет равным 0. Обработка повторного переполнения не предусмотрена, тк она усложнит логику разрабатываемого модуля и увеличит его критический путь (зависит от реализации обработки). Так что будем считать, что двойное переполнение не является ожидаемым поведением и что при первом переполнении последует его программная обработка со стороны пользователя (например, через программу на ассемблере).

При очистке и сбросе обнуляются значения и счётчика, и сигнала переполнения. В таком случае они принимают значение: 0.

Счётчик реагирует только на положительный фронт тактового сигнала или на отрицательный фронт сигнала сброса. Логика модуля устроена так, что она имеет свои приоритеты, и обработка этих событий осуществляется по приоритетам в 3 этапа. Рассмотрим прохождение по этим этапам:

1. Если произошло одно из этих событий, то счётчик проверяет значение сигнала сброса. Если оно в данный момент выставлено в 0 (вне зависимости от 'триггерного' события), то значение счётчика и сигнала переполнения выставляются в 0. На этом работа счётчика заканчивается до следующего события.

2. Если обработка события дошла до этого этапа, то сигнал сброса равен 1. А значит, что произошёл именно положительный фронт тактового сигнала. На этом проверяется значение сигнала очистки. Если оно выставлено в 1, то значения счётчика и сигнала переполнения обнуляются. На этом обработка события заканчивается.

3. На этом этапе происходит стандартное изменение значения счётчика. Если сигнал разрешения счёта выставлен в 1, то значение счётчика увеличивается на 1 (с возможностью переполнения). Иначе – значение счётчика сохраняется.

Заметим, что и сигнал сброса, и сигнал очистки блокируют продолжение увеличения счётчика (вне зависимости от сигнала разрешения счёта). Но они отличаются тем, что сигнал сброса (при отрицательном фронте) мгновенно обнуляет значение счётчика и сигнала переполнения (асинхронно с тактовым сигналом), а сигнал очистки, если он выставлен в 1 – только при положительном фронте тактового сигнала (синхронно с тактовым сигналом).

2.2 Структурная схема

Ниже представлена структурная схема разрабатываемого счётчика. Тонкими стрелками обозначаются шины размером 1 бит. Толстые с одной чертой – N бит. Толстые с двумя черточками – N+1 бит. N - размер счётчика в битах.

Порты, на конце которых присутствуют «i» или «o», будут описаны позже. Порт «cnt_internal» является внутренним и используется для хранения значения счетчика и сигнала переполнения. Его значение в данный момент времени определяют входные порты. Далее этот порт разбивается на выходные порты.

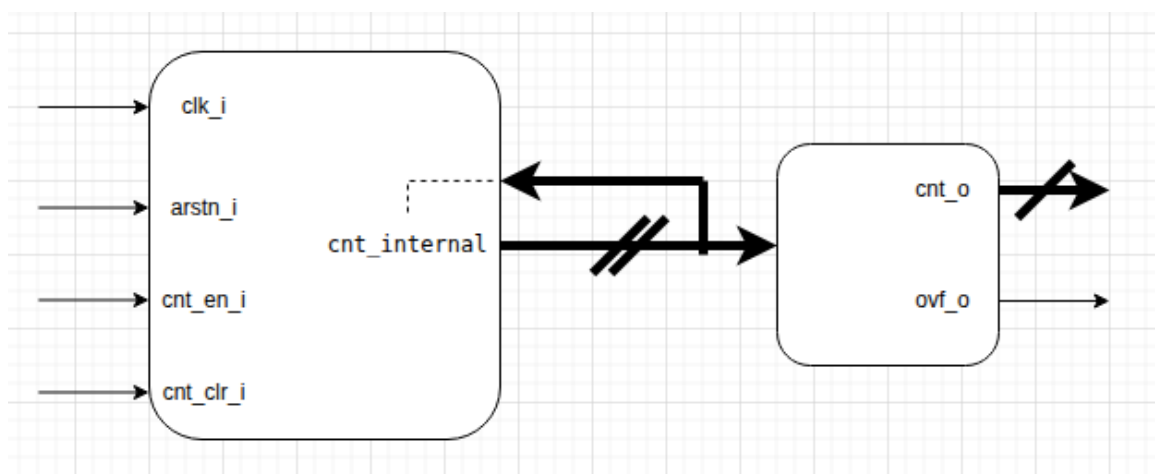


Рис. 1: Структурная схема модуля «cnt_en_clr»

2.3 Параметры

Модуль имеет параметр CNT_WIDTH для указания размера счетчика в битах. Допустимые значения параметра: положительные целые числа (CNT_WIDTH – целое; CNT_WIDTH ≥ 1).

2.4 Порты

Модуль имеет следующие порты:

Название	Ширина	Направление	Описание
clk_i	1	Input	Тактовый сигнал
arstn_i	1	Input	Сигнал сброса
cnt_en_i	1	Input	Сигнал разрешения счёта
cnt_clr_i	1	Input	Сигнал очистки
cnt_o	CNT_WIDTH	Output	Значение счётчика
ovf_o	1	Output	Сигнал переполнения

Таблица 1: Описание портов счетчика

О «clk_i» и «arstn_i» будет рассказано ниже.

«cnt_en_i» – разрешает или запрещает продолжать увеличение счётчика на 1.

«cnt_clr_i» – разрешает или запрещает очистку значений счётчика и сигнала переполнения.

2.5 Тактирование и сброс

Тактирование осуществляется сигналом «clk_i» по положительному фронту.

Сброс осуществляется сигналом «arstn_i» по отрицательному фронту. Т.е., когда значение «arstn_i» меняется с 1 на 0, мгновенно обнуляются значения счетчика и сигнала переполнения и, пока значение «arstn_i» = 0, блокируется вся работа счётчика, а его значение остаётся равным 0 (см. описание приоритетной по этапам реакции счётчика на тактирование и сброс).

2.6 Тестирование

Тестирование будем проводить с помощью двух сценариев: testbench всего функционала счётчика, кроме переполнения и testbench только с переполнением. Более подробно об этих сценариях будет рассказано ниже.

3 Скачивание и запуск кода

Прежде чем мы рассмотрим реализацию модуля на Verilog следует заметить, что для более удобного просмотра кода его можно смотреть на моём [репозитории](#) (код выложен после дедлайна!!!).

Далее идёт описание для скачивания и запуска кода на Linux Ubuntu. Сначала устанавливаем «Icarus Verilog» и «GTKWave»:

```
sudo apt update
sudo apt install iverilog
sudo apt install gtkwave
```

После клонируем репозиторий в свою директорию:

```
git clone https://github.com/Alexandr26Starostin/counter_enable_clear
```

Далее мы будем указывать команды, чтобы компилировать и тестировать с помощью диаграмм наш модуль.

4 Файл с имплементацией разработанного модуля

Ниже представлена реализация модуля «cnt_en_clr» на языке Verilog в соответствии с требованиями:

```
1 //file ./hdl/counter.v
2 //module cnt_en_clr - counter_enable_clear
3 //-----
4
5 //‘define FPGA //-- if we use FPGA, we need in button’s delay (because
6 //button is the clk)
7 //-----
8 ‘ifdef FPGA
9 ‘timescale 1ns / 1ps //for button’s delay
10 ‘endif
11 //-----
12 module cnt_en_clr #(
13     parameter CNT_WIDTH = 8 //size of counter
14 )(
15     input          clk_i,
16     input          arstn_i, //reset - synchronous with clk
17                               // - asynchronous with clk (we
18                               // use it!)
19     input          cnt_en_i,
20     input          cnt_clr_i,
21     output [CNT_WIDTH -1 :0] cnt_o,
22     output          ovf_o
23 );
24
25 reg [CNT_WIDTH :0] cnt_internal;
26 //reg             clr_execute;
27
28 always @(posedge clk_i or negedge arstn_i) begin
29     if (! arstn_i) begin //reset -> //ovf_o == '0 | cnt_o == '0
30                             (immediately after negedge arstn_i)
31         cnt_internal <= {CNT_WIDTH + 1{1'b0}}; //cnt_internal <= '0;
32     end
33
34     else if (cnt_clr_i) begin //clear -> //ovf_o == '0 | cnt_o == '0
35                             (immediately after posedge clk_i)
36         cnt_internal <= {CNT_WIDTH + 1{1'b0}}; //cnt_internal <= '0;
37     end
38
39     else begin
40         cnt_internal <= cnt_en_i ? cnt_internal + 1 : cnt_internal;
41     end
42
43 //-----
44 ‘ifdef FPGA
45 #1000;
46 ‘endif
```

```

45 //-----
46 end
47
48 assign {ovf_o, cnt_o} = cnt_internal;
49
50 endmodule

```

Листинг 1: Содержимое файла ./hdl/counter.v

5 Файлы с testbench для разработанного модуля

Теперь предложим два файла с testbench: один – для всего функционала модуля, другой только для переполнения. О сценариях тестирования будет сказано во время анализа временных диаграмм.

5.1 Файл с testbench для всего функционала модуля, кроме сигнала переполнения

```

1
2 //file ./tb/tb.v
3
4 `timescale 1ns/1ps
5
6 //-----
7 `define GUI_ENABLED
8 //-----
9
10 module cnt_en_clr_tb ();
11
12 //-----
13
14 localparam CNT_WIDTH = 8;
15
16 reg          clk_i;
17 reg          arstn_i;
18 reg          cnt_en_i;
19 reg          cnt_clr_i;
20 wire [CNT_WIDTH-1 :0] cnt_o;
21 wire          ovf_o;
22
23 cnt_en_clr #(
24     .CNT_WIDTH (CNT_WIDTH)
25 ) cnt_en_clr_inst_tb (
26     .clk_i      (clk_i),
27     .arstn_i    (arstn_i),
28
29     .cnt_en_i    (cnt_en_i),
30     .cnt_clr_i   (cnt_clr_i),
31
32     .cnt_o       (cnt_o),
33     .ovf_o       (ovf_o)

```

```

34 );
35
36 //-----
37 //graphic dump
38
39 `ifdef GUI_ENABLED
40 initial begin
41     $dumpvars;
42
43     clk_i      = 1'b0;
44     arstn_i     = 1'b1;
45     cnt_en_i    = 1'b0;
46     cnt_clr_i   = 1'b0;
47     #4;
48
49     clk_i      = 1'b0;
50     arstn_i     = 1'b0;
51     cnt_en_i    = 1'b0;
52     cnt_clr_i   = 1'b0;
53     #6;                                //10
54
55     clk_i      = 1'b1;
56     arstn_i     = 1'b0;
57     cnt_en_i    = 1'b0;
58     cnt_clr_i   = 1'b0;                //20
59     #10
60
61     clk_i      = 1'b0;
62     arstn_i     = 1'b0;
63     cnt_en_i    = 1'b0;
64     cnt_clr_i   = 1'b0;
65     #5
66
67     clk_i      = 1'b0;
68     arstn_i     = 1'b0;
69     cnt_en_i    = 1'b1;
70     cnt_clr_i   = 1'b0;
71     #5                                //30
72
73     clk_i      = 1'b1;
74     arstn_i     = 1'b0;
75     cnt_en_i    = 1'b1;
76     cnt_clr_i   = 1'b0;
77     #4
78
79     clk_i      = 1'b1;
80     arstn_i     = 1'b1;
81     cnt_en_i    = 1'b1;
82     cnt_clr_i   = 1'b0;
83     #6                                //40
84
85     clk_i      = 1'b0;
86     arstn_i     = 1'b1;

```



```

87 cnt_en_i      = 1'b1;
88 cnt_clr_i     = 1'b0;
89 #10                                                    //50
90
91 clk_i         = 1'b1;
92 arstn_i       = 1'b1;
93 cnt_en_i      = 1'b1;
94 cnt_clr_i     = 1'b0;
95 #10                                                    //60
96
97 clk_i         = 1'b0;
98 arstn_i       = 1'b1;
99 cnt_en_i      = 1'b1;
100 cnt_clr_i     = 1'b0;
101 #10                                                    //70
102
103 clk_i         = 1'b1;
104 arstn_i       = 1'b1;
105 cnt_en_i      = 1'b1;
106 cnt_clr_i     = 1'b0;
107 #5
108
109 clk_i         = 1'b1;
110 arstn_i       = 1'b1;
111 cnt_en_i      = 1'b0;
112 cnt_clr_i     = 1'b0;
113 #5                                                    //80
114
115 clk_i         = 1'b0;
116 arstn_i       = 1'b1;
117 cnt_en_i      = 1'b0;
118 cnt_clr_i     = 1'b0;
119 #10                                                    //90
120
121 clk_i         = 1'b1;
122 arstn_i       = 1'b1;
123 cnt_en_i      = 1'b0;
124 cnt_clr_i     = 1'b0;
125 #10                                                    //100
126
127 clk_i         = 1'b0;
128 arstn_i       = 1'b1;
129 cnt_en_i      = 1'b0;
130 cnt_clr_i     = 1'b0;
131 #3
132
133 clk_i         = 1'b0;
134 arstn_i       = 1'b1;
135 cnt_en_i      = 1'b1;
136 cnt_clr_i     = 1'b0;
137 #7                                                    //110
138
139 clk_i         = 1'b1;

```

```

140  arstn_i      = 1'b1;
141  cnt_en_i     = 1'b1;
142  cnt_clr_i    = 1'b0;
143  #10          //120
144
145  clk_i        = 1'b0;
146  arstn_i      = 1'b1;
147  cnt_en_i     = 1'b1;
148  cnt_clr_i    = 1'b0;
149  #6
150
151  clk_i        = 1'b0;
152  arstn_i      = 1'b1;
153  cnt_en_i     = 1'b1;
154  cnt_clr_i    = 1'b1;
155  #4          //130
156
157  clk_i        = 1'b1;
158  arstn_i      = 1'b1;
159  cnt_en_i     = 1'b1;
160  cnt_clr_i    = 1'b1;
161  #10         //140
162
163  clk_i        = 1'b0;
164  arstn_i      = 1'b1;
165  cnt_en_i     = 1'b1;
166  cnt_clr_i    = 1'b1;
167  #7
168
169  clk_i        = 1'b0;
170  arstn_i      = 1'b1;
171  cnt_en_i     = 1'b1;
172  cnt_clr_i    = 1'b0;
173  #3          //150
174
175  clk_i        = 1'b1;
176  arstn_i      = 1'b1;
177  cnt_en_i     = 1'b1;
178  cnt_clr_i    = 1'b0;
179  #10         //160
180
181  clk_i        = 1'b0;
182  arstn_i      = 1'b1;
183  cnt_en_i     = 1'b1;
184  cnt_clr_i    = 1'b0;
185  #10         //170
186
187  clk_i        = 1'b1;
188  arstn_i      = 1'b1;
189  cnt_en_i     = 1'b1;
190  cnt_clr_i    = 1'b0;
191  #10         //180
192

```

```

193 //-----
194
195 clk_i      = 1'b0;
196 arstn_i    = 1'b1;
197 cnt_en_i   = 1'b1;
198 cnt_clr_i  = 1'b0;
199 #10        //190
200
201 clk_i      = 1'b1;
202 arstn_i    = 1'b1;
203 cnt_en_i   = 1'b1;
204 cnt_clr_i  = 1'b0;
205 #10        //200
206
207 //-----
208
209 clk_i      = 1'b0;
210 arstn_i    = 1'b1;
211 cnt_en_i   = 1'b1;
212 cnt_clr_i  = 1'b0;
213 #4
214
215 clk_i      = 1'b0;
216 arstn_i    = 1'b1;
217 cnt_en_i   = 1'b0;
218 cnt_clr_i  = 1'b0;
219 #2
220
221 clk_i      = 1'b0;
222 arstn_i    = 1'b1;
223 cnt_en_i   = 1'b0;
224 cnt_clr_i  = 1'b1;
225 #3        //210
226
227 clk_i      = 1'b1;
228 arstn_i    = 1'b1;
229 cnt_en_i   = 1'b0;
230 cnt_clr_i  = 1'b1;
231 #10        //220
232
233 clk_i      = 1'b0;
234 arstn_i    = 1'b1;
235 cnt_en_i   = 1'b0;
236 cnt_clr_i  = 1'b1;
237 #10        //230
238
239 //-----
240 // $display();
241 // $display
242 // $display ("PASS: 'multibit_adder' test PASSED!");
243 // $display
244 // $display ("-----");

```

```

244     // $display();
245
246     $finish;
247 end
248 `endif
249
250 //-----
251 //generating of tests
252 `ifndef GUI_ENABLED
253 `endif
254
255 //-----
256 //reaction -- automatic verification
257
258 endmodule

```

Листинг 2: Содержимое файла ./tb/tb.v

5.2 Файл с testbench для переполнения

```

1
2 //file ./tb/tb_overflow.v
3
4 `timescale 1ns/1ps
5
6 //-----
7 `define GUI_ENABLED
8 //-----
9
10 module cnt_en_clr_tb ();
11
12 //-----
13
14 localparam CNT_WIDTH = 2; //max_cnt_value = 2**2 - 1 = 3
15
16 reg          clk_i;
17 reg          arstn_i;
18 reg          cnt_en_i;
19 reg          cnt_clr_i;
20 wire [CNT_WIDTH-1 :0] cnt_o;
21 wire          ovf_o;
22
23 cnt_en_clr #(
24     .CNT_WIDTH (CNT_WIDTH)
25 ) cnt_en_clr_inst_tb (
26     .clk_i      (clk_i),
27     .arstn_i    (arstn_i),
28
29     .cnt_en_i    (cnt_en_i),
30     .cnt_clr_i   (cnt_clr_i),
31
32     .cnt_o       (cnt_o),
33     .ovf_o       (ovf_o)

```

```

34 );
35
36 //-----
37 //graphic dump
38
39 `ifdef GUI_ENABLED
40 initial begin
41     $dumpvars;
42
43     //-----
44
45     clk_i      = 1'b1;
46     arstn_i    = 1'b1;
47     cnt_en_i   = 1'b0;
48     cnt_clr_i  = 1'b0;
49     #3
50
51     clk_i      = 1'b1;
52     arstn_i    = 1'b0;
53     cnt_en_i   = 1'b0;
54     cnt_clr_i  = 1'b0;
55     #3
56
57     clk_i      = 1'b1;
58     arstn_i    = 1'b1;
59     cnt_en_i   = 1'b1;
60     cnt_clr_i  = 1'b0;
61     #4          //10
62
63     //0_00
64     //-----
65
66     clk_i      = 1'b0;
67     arstn_i    = 1'b1;
68     cnt_en_i   = 1'b1;
69     cnt_clr_i  = 1'b0;
70     #10         //20
71
72     clk_i      = 1'b1;
73     arstn_i    = 1'b1;
74     cnt_en_i   = 1'b1;
75     cnt_clr_i  = 1'b0;
76     #10         //30
77
78     //0_01
79     //-----
80
81     //-----
82
83     clk_i      = 1'b0;
84     arstn_i    = 1'b1;
85     cnt_en_i   = 1'b1;
86     cnt_clr_i  = 1'b0;

```

```

87      #10                                //40
88
89      clk_i      = 1'b1;
90      arstn_i     = 1'b1;
91      cnt_en_i    = 1'b1;
92      cnt_clr_i   = 1'b0;
93      #10                                //50
94
95      // =0_10
96      // -----
97
98
99      // -----
100
101      clk_i      = 1'b0;
102      arstn_i     = 1'b1;
103      cnt_en_i    = 1'b1;
104      cnt_clr_i   = 1'b0;
105      #10                                //60
106
107      clk_i      = 1'b1;
108      arstn_i     = 1'b1;
109      cnt_en_i    = 1'b1;
110      cnt_clr_i   = 1'b0;
111      #10                                //70
112
113      // =0_11
114      // -----
115
116      // -----
117
118      clk_i      = 1'b0;
119      arstn_i     = 1'b1;
120      cnt_en_i    = 1'b1;
121      cnt_clr_i   = 1'b0;
122      #10                                //80
123
124      clk_i      = 1'b1;
125      arstn_i     = 1'b1;
126      cnt_en_i    = 1'b1;
127      cnt_clr_i   = 1'b0;
128      #10                                //90
129
130      // =1_00
131      // -----
132
133      // -----
134
135      clk_i      = 1'b0;
136      arstn_i     = 1'b1;
137      cnt_en_i    = 1'b1;
138      cnt_clr_i   = 1'b0;
139      #10                                //100

```

```

140
141 clk_i      = 1'b1;
142 arstn_i    = 1'b1;
143 cnt_en_i   = 1'b1;
144 cnt_clr_i  = 1'b0;
145 #10                //110
146
147 //1_01
148 //-----
149
150 //-----
151
152 clk_i      = 1'b0;
153 arstn_i    = 1'b1;
154 cnt_en_i   = 1'b1;
155 cnt_clr_i  = 1'b0;
156 #10                //120
157
158 clk_i      = 1'b1;
159 arstn_i    = 1'b1;
160 cnt_en_i   = 1'b1;
161 cnt_clr_i  = 1'b0;
162 #10                //130
163
164 //1_10
165 //-----
166
167 //-----
168
169 clk_i      = 1'b0;
170 arstn_i    = 1'b1;
171 cnt_en_i   = 1'b1;
172 cnt_clr_i  = 1'b0;
173 #10                //140
174
175 clk_i      = 1'b1;
176 arstn_i    = 1'b1;
177 cnt_en_i   = 1'b1;
178 cnt_clr_i  = 1'b0;
179 #10                //150
180
181 //1_11
182 //-----
183
184 //-----
185
186 clk_i      = 1'b0;
187 arstn_i    = 1'b1;
188 cnt_en_i   = 1'b1;
189 cnt_clr_i  = 1'b0;
190 #10                //160
191
192 clk_i      = 1'b1;

```

```

193 arstn_i      = 1'b1;
194 cnt_en_i     = 1'b1;
195 cnt_clr_i    = 1'b0;
196 #10                                //170
197
198 // =0_00
199 // -----
200
201 // -----
202
203 clk_i        = 1'b0;
204 arstn_i      = 1'b1;
205 cnt_en_i     = 1'b1;
206 cnt_clr_i    = 1'b0;
207 #10                                //180
208
209 clk_i        = 1'b1;
210 arstn_i      = 1'b1;
211 cnt_en_i     = 1'b1;
212 cnt_clr_i    = 1'b0;
213 #10                                //190
214
215 // =0_01
216 // -----
217
218 $finish;
219 end
220 `endif
221
222 endmodule

```

Листинг 3: Содержимое файла ./tb/tb_overflow.v

6 Временные диаграммы

Все диаграммы можно посмотреть в директории «./pictures».

6.1 testbench для всего функционала модуля, кроме сигнала переполнения

Чтобы увидеть временную диаграмму «dump_1.png» или «dump_1.jpg», выполняем следующие команды:

```

iverilog ./hdl/counter.v ./tb/tb.v -g2005-sv -o counter
./counter
gtkwave dump.vcd

```

В открытой программе нажимаем «cnt_en_clr_tb» и внизу выбираем нужные нам порты и нажимаем «append». Далее регулируем масштаб диаграммы и видим:

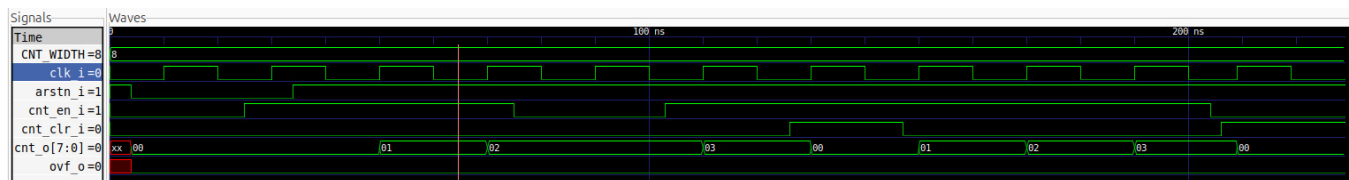


Рис. 2: Временная диаграмма с testbench для всего функционала модуля

Проанализируем полученную диаграмму «dump_1.png» или «dump_1.jpg».

Параметр CNT_WIDTH равен 8.

Цена деления синих штрихов составляет 10 нс. Вся симуляция длится суммарно 230 нс.

Сначала значения счётчика и сигнала переполнения не определены однозначно. Поэтому в момент времени **4 нс** происходит сброс счётчика и сигнала переполнения и их значения обнуляются (отрицательный фронт arstn_i). В течение всей симуляции значение ovf_o будет равно 0.

В моменты времени **10 нс** (cnt_en_i == 0) и **30 нс** (cnt_en_i == 1) показывается, что сигнал сброса arstn_i является самым приоритетным (1-ый по приоритету) и вне зависимости от cnt_en_i значение счётчика остаётся нулевым даже при положительном фронте clk_i.

В **50 нс** выставлены в 1 arstn_i (нет сброса) и cnt_en_i (разрешён счёт). В этот момент по положительному фронту clk_i происходит увеличение значения счётчика. Аналогично происходит в момент времени **70 нс**.

В **90 нс** cnt_en_i опускается, т.е. становится равным 0 (запрёт счёта), и по положительному фронту clk_i не происходит увеличение счётчика.

Далее в **110 нс** значение cnt_en_i становится равным 1 и снова счётчик увеличивается.

В **126 нс** выставляется в 1 cnt_clr_i. И в моменты времени **130 нс** (cnt_en_i == 1) и **210 нс** (cnt_en_i == 0) показывается, сигнал очистки более приоритетный (имеет 2 приоритет), чем сигнал разрешения счёта (имеет 3 приоритет), и что вне зависимости от cnt_en_i, если выставлен в 1 cnt_clr_i, то по положительному фронту clk_i происходит обнуление значения счётчика и сигнала переполнения. В моменты времени **150 нс**, **170 нс** и **190 нс** просто происходит увеличение значения счётчика (как до этого).

На этом симуляция заканчивается. Мы проверили весь функционал модуля, кроме переполнения, показали работу приоритетов при обработке событий и посмотрели, как работает счётчик.

6.2 Файл с testbench для переполнения

Теперь посмотрим, как происходит переполнение счётчика.

Чтобы увидеть временную диаграмму «dump_2.png», выполняем следующие команды:

```
iverilog ./hdl/counter.v ./tb/tb_overflow.v -g2005-sv -o counter
./counter
gtkwave dump.vcd
```

В открытой программе нажимаем «cnt_en_clr_tb» и внизу выбираем нужные нам порты и нажимаем «append». Далее регулируем масштаб диаграммы и видим:

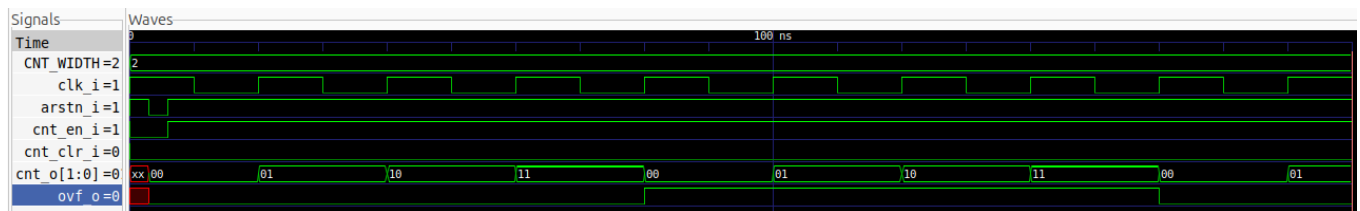


Рис. 3: Временная диаграмма с testbench для переполнения

Проанализируем полученную диаграмму «dump_2.png».

Параметр CNT_WIDTH равен 2.

Цена деления синих штрихов составляет 10 нс. Вся симуляция длится суммарно 190 нс.

В симуляции так же как и до этого происходит сброс счётчика. Дальше каждые 20 нс происходит увеличение счётчика на 1.

Самыми интересными являются моменты времени **80 нс** (первое переполнение) и **160 нс** (второе переполнение). Как мы видим, при каждом переполнении значение ovf_o меняется на противоположное, а значение счётчика cnt_o обнуляется. Напомним, что мы считаем второе переполнение не ожидаемым поведением. Но мы были обязаны показать, как оно происходит.

Итак, теперь наш модуль прошёл полную проверку работы с помощью временных диаграмм.

7 Вывод

Мы разработали модуль cnt_en_clr, описали его функционал, составили документацию к нему, реализовали его на Verilog, протестировали с помощью временных диаграмм. В итоге у нас получился полностью рабочий счётчик с разрешением на счёт и очистку. Единственной возможной проблемой является возможность второго переполнения. Поскольку в техническом задании не было чётких требований по этому поводу, мы решили, что пользователь сам должен контролировать переполнения и, если ему это нужно, не допускать возникновения второго. При необходимости, мы можем дописать логику модуля так, чтобы второе переполнение не возникало.